

Homi: Projeto e Implementação de uma Linguagem de Domínio Específico e seu Compilador para Automações do Home Assistant

Trabalho Final — Disciplina de Compiladores (2026)

Ana Lilian Alfonso Toledo¹, Tobias Viero de Oliveira¹

¹Curso de Ciência da Computação (UFSM)

1. Introdução

A plataforma *Home Assistant* é um dos sistemas de automação residencial de código aberto mais difundidos. Suas automações são descritas em arquivos YAML que combinam gatilhos (*triggers*), condições (*conditions*) e ações (*actions*). Embora poderoso, esse formato é verboso, sensível à indentação e cheio de armadilhas para usuários sem formação em computação: aspas obrigatórias em certos valores, nomes de serviços específicos por domínio (`light.turn_on`, `switch.turn_off`) e uma hierarquia de chaves difícil de memorizar.

O objetivo deste trabalho é projetar uma linguagem de domínio específico (DSL), chamada Homi, que abstraia essa complexidade. Em Homi, o usuário escreve a automação em português estruturado e legível, e o compilador (tradutor) gera automaticamente o YAML correto. O projeto cobre rigorosamente as fases de *front-end* de um compilador, além do início do *back-end* (geração de código intermediário).

A linguagem foi desenhada a partir de um conjunto de 29 automações YAML reais, fornecidas como base, que serviram tanto de inspiração para as construções da linguagem quanto de oráculo para validar a saída do compilador.

1.1. Organização da arquitetura

O compilador percorre quatro fases clássicas, cada uma implementada em um módulo Python independente, conforme ilustra a Figura 1. A implementação léxica e sintática usa a biblioteca PLY (*Python Lex-Yacc*), que gera automaticamente o DFA do *scanner* e a tabela LALR(1) do *parser* a partir de especificações declarativas.

2. Definição da Linguagem Homi

2.1. Princípios de projeto

A linguagem foi orientada por quatro princípios voltados ao público leigo:

- **Palavras-chave em português** (quando, se, faça, ligar, aguardar), aproximando o código da linguagem natural.
- **Estrutura declarativa** com seções claramente nomeadas e ordenadas: quando (gatilhos), se (condições), faça (ações) e modo (modo de execução).
- **Concordância de gênero**: estados aceitam as formas masculina e feminina (desligado/desligada, armado/armada), o que torna as condições gramaticalmente naturais (“a luz está desligada”).
- **Compatibilidade**: os estados em inglês (`on`, `off`) também são aceitos, para usuários já familiarizados com o Home Assistant.

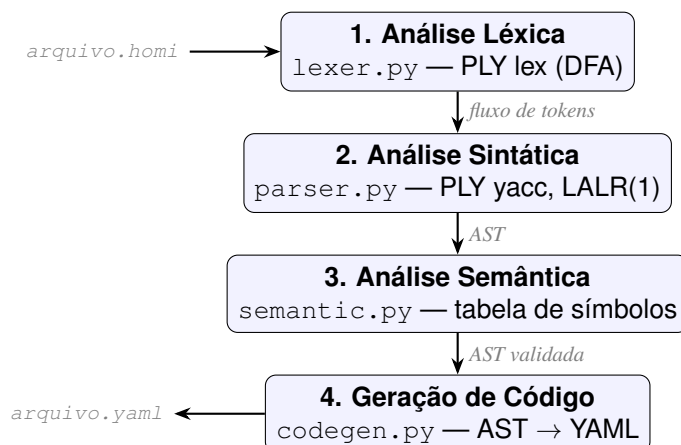


Figura 1. Pipeline de compilação do Homi: das quatro fases, três pertencem ao *front-end* e a quarta ao início do *back-end*.

2.2. Estrutura de um script Homi

Um programa Homi é uma sequência de uma ou mais automações. Cada automação tem a forma geral mostrada na Listagem 1. A seção quando é obrigatória; se, faça e modo são opcionais, mas, quando presentes, devem respeitar essa ordem.

Listing 1. Estrutura geral de um script Homi.

[illegible]

3. Especificação da GLC (Gramática Livre de Contexto)

A gramática da linguagem Homi foi especificada formalmente e implementada na forma bottom-up LR, mais precisamente LALR(1), por meio de tabela de parsing gerada automaticamente pelo PLY. A gramática final possui 70 produções, 52 terminais e produz uma tabela LALR(1) com 130 estados e nenhum conflito *shift-reduce* ou *reduce-reduce*, conforme verificado no arquivo `parser.out` gerado pela ferramenta. A ausência de conflitos foi um objetivo de projeto, alcançado ao se modelar os sufixos opcionais (por exemplo como `STRING,brilho NUMBER %`) e as seções opcionais por meio de não-terminais dedicados (`opt_se_section`, etc.) em vez de alternativas inline.

3.1. Terminais

As Tabelas 1, 2 e 3 discriminam, respectivamente, as palavras reservadas, os *tokens* de estado e os *tokens* literais reconhecidos pelo analisador léxico.

Tabela 1. Palavras reservadas da linguagem Homi.

Token	Lexema	Significado
AUTOMACAO	automacao	Declaração de automação
QUANDO	quando	Seção de gatilhos
SE	se	Condições / ação condicional
FACA	faca	Seção de ações
MODO	modo	Modo de execução
LIGAR	ligar	Ação: ligar entidade
DESLIGAR	desligar	Ação: desligar entidade
AGUARDAR	aguardar	Ação: aguardar tempo
NOTIFICAR	notificar	Ação: enviar notificação
PARA	para	Preposição (alvo da notificação)
ENTAO	entao	Bloco <i>then</i>
SENAO	senao	Bloco <i>else</i>
FIM	fim	Fecha bloco <i>se/escolher</i>
ESCOLHER	escolher	Ação: escolha múltipla
CASO	caso	Alternativa em <i>escolher</i>
ESTA	esta	Predicado “está” (condições)
MUDA_PARA	muda_para	Evento de mudança de estado
COMO	como	Define o <i>id</i> do gatilho
SENSOR	sensor	Tipo: sensor
LUZ	luz	Tipo: luz
INTERRUPTOR	interruptor	Tipo: <i>switch</i>
ALARME	alarme	Tipo: painel de alarme
MEDIA	media	Tipo: <i>media player</i>
TIMER	timer	Tipo: <i>timer</i>
HORARIO	horario	Palavra-chave de tempo
AO_HORARIO	ao_horario	Gatilho de horário exato
ENTRE / E	entre / e	Intervalo de tempo
NAScer_DO_SOL	nascerc_do_sol	Gatilho: <i>sunrise</i>
POR_DO_SOL	por_do_sol	Gatilho: <i>sunset</i>
OFFSET	offset	Deslocamento solar
BRILHO	brilho	Parâmetro de brilho
UNICO / REINICIAR	unico / reiniciar	Modos <i>single / restart</i>
FILA / PARALELO	fila / paralelo	Modos <i>queued / parallel</i>

Tabela 2. Tokens de estado (com formas PT e EN).

Token	Lexemas aceitos	Significado
ON / OFF	on / off	Ligado / desligado (EN)
LIGADO	ligado, ligada	Estado ligado (PT)
DESLIGADO	desligado, desligada	Estado desligado (PT)
ARMADO	armado, armada	Estado armado (alarme)
DESARMADO	desarmado, desarmada	Estado desarmado (alarme)

Tabela 3. Tokens literais e suas expressões regulares.

Token	Expressão regular	Exemplos
ENTITY_ID	[a-z_][a-z0-9_]*\.[a-z_][a-z0-9_.*]	light.sala
TIME	\d{2}:\d{2}(:\d{2})?	06:30,23:00:00
DURACAO	-?\d+h\d+ -?\d+(ms s min h)	45s,-1h30
NUMBER	\d+(\.\d+)?	80,3.14
STRING	"[^"\n]*" '[^'\n]*'	"Movimento!"
PORCENTO	%	%
LBRACE / RBRACE / COLON	\{ / \} / :	{ } :

3.2. Não-terminais e produções

A Listagem 2 apresenta a gramática completa. A notação usa MAIÚSCULAS para terminais, *minúsculas* para não-terminais, | para alternativas e ϵ para a produção vazia.

Listing 2. Gramática Livre de Contexto da linguagem Homi.

program	: automation_list
automation_list	: automation automation_list automation
automation	: AUTOMACAO STRING LBRACE body RBRACE
body	: when_section opt_se_section opt_faca_section opt_modos_section
when_section	: QUANDO COLON trigger_list
trigger_list	: trigger trigger_list trigger trigger_list error trigger // recuperacao
trigger	: domain_kw ENTITY_ID MUDA_PARA state_value domain_kw ENTITY_ID MUDA_PARA state_value COMO STRING AO_HORARIO TIME AO_HORARIO TIME COMO STRING POR_DO_SOL POR_DO_SOL COMO STRING POR_DO_SOL OFFSET DURACAO POR_DO_SOL OFFSET DURACAO COMO STRING NASCER_DO_SOL NASCER_DO_SOL COMO STRING NASCER_DO_SOL OFFSET DURACAO NASCER_DO_SOL OFFSET DURACAO COMO STRING
domain_kw	: SENSOR LUZ INTERRUPTOR ALARME MEDIA TIMER
opt_se_section	: se_section (epsilon)
se_section	: SE COLON condition_list
condition_list	: condition condition_list condition

	condition_list error condition // recuperacao
condition	: ENTITY_ID ESTA state_value domain_kw ENTITY_ID ESTA state_value HORARIO ENTRE TIME E TIME
opt_faca_section	: faca_section (epsilon)
faca_section	: FACA COLON action_list
action_list	: action action_list action action_list error action // recuperacao
action	: LIGAR ENTITY_ID LIGAR ENTITY_ID BRILHO NUMBER PORCENTO DESLIGAR ENTITY_ID AGUARDAR DURACAO NOTIFICAR STRING PARA ENTITY_ID SE condition_list ENTAO COLON action_list FIM SE condition_list ENTAO COLON action_list SENAO COLON action_list FIM ESCOLHER COLON choice_list FIM
choice_list	: choice choice_list choice
choice	: CASO condition_list FACA COLON action_list
opt_modos_section	: modos_section (epsilon)
modos_section	: MODO COLON modos_value
modos_value	: UNICO REINICIAR FILA PARALELO
state_value	: ON OFF LIGADO DESLIGADO ARMADO DESARMADO STRING NUMBER

4. Análise Léxica (Scanner)

4.1. Implementação

O analisador léxico (`lexer.py`) foi implementado com `PLY lex`, que constrói automaticamente um Autômato Finito Determinístico (DFA) a partir das expressões regulares declaradas nas funções `t_*`. As palavras reservadas são reconhecidas como o padrão genérico de identificador e, em seguida, classificadas por consulta a um dicionário `reserved` — técnica padrão que evita uma regra de expressão regular por palavra-chave.

4.2. Prioridade entre tokens

Em `PLY`, funções-*token* têm prioridade sobre *tokens* declarados como *strings*, e funções são testadas na ordem de definição no arquivo. Essa ordem foi cuidadosamente escolhida para resolver ambiguidades léxicas, como resume a Tabela 4.

Tabela 4. Ordem de prioridade dos *tokens* complexos e sua justificativa.

#	Token	Justificativa da prioridade
1	ENTITY_ID	Testado antes de ID para que <code>light.sala</code> seja um único <i>token</i> , e não <code>light + erro</code> .
2	DURACAO	Testado antes de NUMBER para que <code>45s</code> não vire <code>45</code> seguido de erro.
3	TIME	Testado antes de COLON para que <code>01:15</code> não vire <code>01 + : + 15</code> .
4	NUMBER	Números genéricos (brilho, estados numéricos).
5	STRING	Literais entre aspas simples ou duplas.
6	ID	Resolve palavras reservadas via dicionário; o que não for reservado é erro léxico.

4.3. Comentários e contagem de linhas

Comentários de linha (`// . . .`) são descartados. Espaços, tabulações e retornos de carro são ignorados. As quebras de linha incrementam o contador `lineno` do *lexer*, base para o reporte preciso de erros com o número da linha, como exigido pelos requisitos.

Listing 3. Tratamento de comentários e linhas.

```
def t_COMMENT(t):
    r'//[^\n]*'
    pass # descartado

def t_newline(t):
    r'\n+'
    t.lexer.lineno += len(t.value)
```

4.4. Tratamento de erros léxicos

A detecção de erros léxicos ocorre em dois pontos: (i) `t_error`, acionado por um caractere isolado inválido (por exemplo `@`, `#`), que registra o erro, avança um caractere (`lexer.skip(1)`) e continua; e (ii) `t_ID`, quando um identificador sem ponto não consta do dicionário `reserved`, registrando o erro e descartando o *token*. Todos os erros são acumulados na lista `lexer.lex_errors` para exibição consolidada — o *scanner* não aborta no primeiro erro.

5. Análise Sintática (Parser)

5.1. Método: LALR(1) baseado em tabela

Conforme permitido pelos requisitos (forma *bottom-up* LR(k)), o analisador sintático (`parser.py`) é implementado com `PLY yacc`, que constrói a tabela LALR(1) a partir das funções `p_*`. A tabela é materializada em `parsetab.py` e o relatório de estados em `parser.out`. Como mencionado na Seção 3, a gramática gera uma tabela sem conflitos.

5.2. Construção da AST

Cada produção constrói nós da Árvore Sintática Abstrata (AST), definidos como *data-classes* em `ast_nodes.py`. A Tabela 5 resume o mapeamento.

Tabela 5. Produções principais e nós da AST gerados.

Produção	Nó da AST
automation	AutomationNode(name, triggers, conditions, actions, mode)
trigger	TriggerNode(kind, entity, to, at, sun_event, offset, id)
condition	ConditionNode(kind, entity, state, after, before)
action	ActionNode(kind, entity, duration, message, brightness, ...)
choice	ChoiceNode(conditions, actions)

5.3. Recuperação de erros — Modo Pânico

Atendendo ao requisito de não abortar no primeiro erro, o *parser* implementa recuperação em modo pânico por meio de três produções de erro do PLY, uma para cada lista repetível:

Listing 4. Produções de recuperação de erro (modo pânico).

trigger_list	:	trigger_list	error trigger
condition_list	:	condition_list	error condition
action_list	:	action_list	error action

Quando um *token* inesperado surge dentro de uma lista, o PLY descarta *tokens* (sincronizando) até encontrar algo que case com o próximo item da lista, registrando uma mensagem como “*gatilho inválido ignorado (modo pânico)*” e permitindo que o restante do arquivo continue sendo analisado. Erros isolados são tratados em `p_error`, que reporta o *token* inesperado com seu tipo e a linha.

6. Análise Semântica

6.1. Tabela de Símbolos

A análise semântica (`semantic.py`) baseia-se em uma **tabela de símbolos** (`SymbolTable`) que mapeia o *prefixo* de cada `entity_id` — isto é, o domínio do Home Assistant — a um tipo semântico `Homi`. A Tabela 6 mostra um recorte desse mapeamento.

Tabela 6. Mapeamento domínio HA → tipo Homi (recorte).

Domínio Home Assistant	Tipo Homi
light	luz
switch, input_boolean, fan	interruptor
binary_sensor, sensor, weather, climate	sensor
alarm_control_panel	alarme
timer	timer
media_player	media
cover	cortina
notify	notificacao

6.2. Regras de verificação

A verificação de tipos e a consistência externa (validar que os serviços são compatíveis com o domínio da entidade) são realizadas por um conjunto de regras aplicadas durante a travessia da AST, resumidas na Tabela 7.

Tabela 7. Regras semânticas verificadas pelo compilador.

#	Regra	Exemplo de erro
1	O domínio do <code>entity_id</code> deve existir na tabela.	<code>xyz.sensor</code> esta on
2	Estados de alarme (<code>disarmed</code> , <code>armed_*</code>) só em <code>alarm_control_panel</code> .	<code>luz light.sala</code> esta desarmada
3	brilho é exclusivo do domínio <code>light</code> .	<code>ligar switch.x</code> brilho 80%
4	O alvo de notificar deve ser <code>notify.*</code> .	notificar "m" para <code>light.sala</code>
5	A duração de aguardar não pode ser negativa.	aguardar -5s
6	Nome de automação duplicado (aviso, não erro).	<code>duas automacao</code> "X"

6.3. Normalização de estados

Durante a análise sintática, os estados em português são normalizados para os valores canônicos do Home Assistant, de modo que a fase semântica e a geração de código operem sobre um vocabulário único: `ligado/ligada/on` → `on`; `desligado/desligada/off` → `off`; `armado/armada` → `armed_home`; `desarmado/desarmada` → `disarmed`.

7. Geração de Código Intermediário (YAML)

7.1. Mapeamento AST → YAML

A última fase (`codegen.py`) percorre a AST validada e produz a estrutura declarativa do Home Assistant. A Tabela 8 apresenta o mapeamento das principais construções.

Tabela 8. Mapeamento das construções Homi para YAML do Home Assistant.

Construção Homi	YAML gerado
automacao "Nome" {...}	- alias: Nome
sensor X muda_para on como "ID"	trigger: state, to: 'on', id: ID
ao_horario 05:00	trigger: time, at: '05:00:00'
por_do_sol offset -1h30	trigger: sun, event: sunset, offset: '-01:30:00'
X esta desarmado	condition: state, state: disarmed
horario entre 01:00 e 06:30	condition: time, after/before
ligar light.X	action: light.turn_on, target
ligar light.X brilho 50%	data: {brightness_pct: 50}
desligar switch.X	action: switch.turn_off
aguardar 1h30	delay: {hours: 1, minutes: 30}
notificar "m" para notify.X	action: notify.X, data: {message: m}
se ... entao: ...	if/then/else
senao: ... fim	
escolher: caso ... fim	choose: [{conditions, sequence}]
modo: reiniciar	mode: restart

7.2. Indentação e tipagem do YAML

O YAML do Home Assistant é sensível à indentação e interpreta certos escalares de forma indesejada: `on`, `off`, `yes` e `no` viram booleanos, e horários como `05:00:00` podem ser lidos como sexagesimais. Para evitar isso, a geração usa PyYAML com um *dumper* customizado (`_HomiDumper`) que força aspas simples nesses casos, garantindo um YAML sintaticamente válido e semanticamente correto para a plataforma.

8. Exemplos: scripts Homi e YAMLs resultantes

Esta seção apresenta exemplos reais processados pelo compilador. As saídas YAML mostradas foram produzidas pela execução efetiva da ferramenta.

8.1. Exemplo 1 — Luz por movimento com lógica condicional de horário

Listing 5. Entrada: `corredor_movimento.homi`.

```

automacao "Corredor - movimento" {
  quando:
    sensor binary_sensor.motion_sensor_movimento muda_para on como "Detectou"
    sensor binary_sensor.corredor_suite_luminance muda_para on como "Detectou2"
    sensor binary_sensor.tz3000_6ygjfy1l_ts0202 muda_para on como "Detectou3"

  se:
    alarme alarm_control_panel.alarmo esta desarmado
    luz light.corda_led_corredor esta desligada

  faca:

```

```

    ligar light.corda_led_corredor
    se
        horario entre 01:15 e 12:00
    entao:
        aguardar 45s
    senao:
        aguardar 1min
    fim
    desligar light.corda_led_corredor

modo: reiniciar
}

```

Listing 6. Saída: corredor_movimento.yaml (gerada pelo compilador).

```

- alias: Corredor - movimento
  triggers:
  - trigger: state
    entity_id: binary_sensor.motion_sensor_movimento
    to: 'on'
    id: Detectou
  - trigger: state
    entity_id: binary_sensor.corredor_suite_luminance
    to: 'on'
    id: Detectou2
  - trigger: state
    entity_id: binary_sensor.tz3000_6ygjfyll_ts0202
    to: 'on'
    id: Detectou3
  conditions:
  - condition: state
    entity_id: alarm_control_panel.alarmo
    state: disarmed
  - condition: state
    entity_id: light.corda_led_corredor
    state: 'off'
  actions:
  - action: light.turn_on
    target:
      entity_id: light.corda_led_corredor
  - if:
    - condition: time
      after: '01:15:00'
      before: '12:00:00'
    then:
      - delay:
          seconds: 45
    else:
      - delay:
          minutes: 1
  - action: light.turn_off
    target:
      entity_id: light.corda_led_corredor
  mode: restart

```

8.2. Exemplo 2 — Gatilho solar com bloco de escolhas

Listing 7. Entrada: por_do_sol.homi (primeira automação).

```

automacao "Por do Sol" {
  quando:
    por_do_sol offset -1h30 como "chuva"
    por_do_sol offset -1h15 como "nublado"
    por_do_sol offset -45min como "sol"

  se:
    alarme alarm_control_panel.alarmo esta desarmado

```

```
faca:
  escolher:
    caso
      interruptor switch.luzes_da_cozinha esta desligado
    faca:
      ligar switch.luzes_da_cozinha
    caso
      interruptor switch.luzes_da_sala esta desligado
    faca:
      ligar switch.luzes_da_sala
  fim

modo: unico
}
```

Listing 8. Saída: por_do_sol.yaml (trecho da primeira automação).

```
- alias: Por do Sol
  triggers:
  - trigger: sun
    event: sunset
    offset: '-01:30:00'
    id: chuva
  - trigger: sun
    event: sunset
    offset: '-01:15:00'
    id: nublado
  - trigger: sun
    event: sunset
    offset: '-00:45:00'
    id: sol
  conditions:
  - condition: state
    entity_id: alarm_control_panel.alarmo
    state: disarmed
  actions:
  - choose:
    - conditions:
      - condition: state
        entity_id: switch.luzes_da_cozinha
        state: 'off'
      sequence:
      - action: switch.turn_on
        target:
          entity_id: switch.luzes_da_cozinha
    - conditions:
      - condition: state
        entity_id: switch.luzes_da_sala
        state: 'off'
      sequence:
      - action: switch.turn_on
        target:
          entity_id: switch.luzes_da_sala
  mode: single
```

8.3. Exemplo 3 — Detecção e reporte de erros

8.3.1. Erros léxicos

A entrada `erro_lexico.homi` contém os caracteres inválidos `@` e `#`. O compilador reporta cada ocorrência com sua linha e prossegue, sem abortar:

Listing 9. Saída do compilador para erros léxicos (trecho).

```
Erro lexico [linha 7] identificador desconhecido 'binary_sensor'
```

```

Erro lexico [linha 7] caractere invalido '.'
Erro lexico [linha 7] caractere invalido '@'
Erro lexico [linha 7] identificador desconhecido 'motion'
Erro lexico [linha 10] caractere invalido '#'
...

```

8.3.2. Erros sintáticos (modo pânico)

A entrada `erro_sintatico.homi` contém uma automação sem a seção obrigatória quando e outra com um bloco `se...entao` sem o fechamento `fim`. Graças ao modo pânico, o *parser* reporta todos os problemas em vez de parar no primeiro:

Listing 10. Saída do compilador para erros sintáticos (trecho).

```

Erro sintatico [linha 6] token inesperado 'esta' (tipo: ESTA)
Erro sintatico [linha 8] token inesperado 'faca' (tipo: FACA)
Erro sintatico [linha 11] token inesperado 'modo' (tipo: MODO)
...
Erro sintatico fim de arquivo inesperado
Compilacao interrompida: 22 erro(s) lexico(s)/sintatico(s).

```

8.3.3. Erros semânticos

A entrada `erro_semantico.homi` viola três regras de tipos. O compilador identifica cada violação, com linha e descrição precisa:

Listing 11. Saída do compilador para erros semânticos.

```

Erro semantico [linha 9] 'brilho' so pode ser usado com entidades do
    dominio 'light'; 'switch.luzes_da_sala' pertence ao dominio 'switch'
Erro semantico [linha 22] o alvo de 'notificar' deve ter dominio 'notify';
    recebido: 'light.sala' (dominio: 'light')
Erro semantico [linha 35] estado 'disarmed' e valido apenas para
    'alarm_control_panel'; 'light.sala' tem dominio 'light'
Compilacao interrompida: 3 erro(s) semantico(s).

```

9. Estrutura do Projeto e Instruções de Compilação

O repositório é organizado em módulos coesos, um por fase, com diretórios de documentação e testes, conforme a Listagem 12. A compilação e a execução são automatizadas por um Makefile com os alvos `install`, `run`, `test` e `clean`.

Listing 12. Estrutura do repositório.

```

homi_compiler/
  Makefile          # alvos: run, test, clean, install
  homi.py           # ponto de entrada (CLI)
  lexer.py          # analise lexica (PLY lex)
  parser.py         # analise sintatica (PLY yacc, LALR(1))
  ast_nodes.py      # nos da AST (dataclasses)
  semantic.py       # analise semantica + tabela de simbolos
  codegen.py        # geracao de codigo YAML
  errors.py         # ErroLexico, ErroSintatico, ErroSemantico
  parsetab.py       # tabela LALR(1) gerada pelo PLY
  docs/             # gramatica.md
  tests/
    exemplos_validos/ # *.homi + *.yaml esperado
    exemplos_invalidos/ # erro_lexico/sintatico/semantico.homi

```

Listing 13. Comandos principais.

```
make install      # pip install ply pyyaml pytest
make run FILE=tests/exemplos_validos/corredor_movimento.homi
python homi.py minha_automacao.homi -o resultado.yaml
python homi.py minha_automacao.homi --check    # so verifica erros
python homi.py minha_automacao.homi --ast      # imprime a AST
make test
```

10. Conclusão

Este trabalho apresentou o projeto e a implementação completa do compilador da linguagem **Homi**, cobrindo as três fases do *front-end* (análise léxica com DFA, análise sintática LALR(1) e análise semântica com tabela de símbolos e verificação de tipos) e o início do *back-end* (geração de código intermediário em YAML). Todos os requisitos obrigatórios foram atendidos: reconhecimento de *tokens* complexos (*entity_id*, unidades de tempo, *strings* e operadores), contagem de linhas e tratamento de comentários; implementação baseada em tabela de *parsing* com recuperação de erros em modo pânico; tabela de símbolos com verificação de tipos e consistência externa; e tradução fiel para a estrutura YAML, com tratamento da indentação e da tipagem de escalares.

A principal contribuição de projeto é a abstração da complexidade do Home Assistant em uma sintaxe próxima da linguagem natural em português, com suporte a concordância de gênero — o que torna a linguagem efetivamente acessível ao público-alvo de usuários leigos. Como trabalhos futuros, destacam-se a expansão do conjunto de serviços suportados (por exemplo, *media players* e cortinas com parâmetros), variáveis e *templates*, e a validação de *entity_id* contra o registro real de entidades de uma instalação.